

PHYS7101: NUMERICAL METHODS AND DATA ANALYSIS · LECTURE 3

Solving equations and fitting models

Mathew Smith & Aneta Stefanovska

From numerical solutions to data

Where we left off

From Lecture 2

- Time stepping: order of accuracy; verify convergence
- Six questions before you simulate
- Static problems, $f(x) = 0$, need root finding
- Time variability can look like noise
- Structured residuals: the model misses something
- **Verification** (model solved correctly) and **validation** (model represents the system)

Today

- Solving $f(x) = 0$ robustly, in one and many dimensions
- Finding minima: optimisation
- Fitting models to data: least squares
- Judging a fit: residuals, χ^2 , uncertainties
- Testing the fit on generated data

Validation needs data — and first we must **estimate the model's parameters** from the data.

One chain of ideas

Step (Part)	Question	What it means
1. Root finding (A)	For which x is $f(x) = 0$?	Iterate: bisection, Newton
2. Optimisation (A)	For which x is $f(x)$ smallest?	Find the points where $\nabla f = 0$ (root finding), then check which is a minimum
3. Fitting (B)	Which parameters bring the model closest to the data?	Minimise $\sum(\text{data} - \text{model})^2$: optimisation
4. Goodness of fit (C)	Does the model describe the data?	Residuals: no structure, size of the errors
5. Testing (D)	Does the fit recover a known answer?	Fit generated data, then measured data

Steps 1–3 **build on each other**: a fit is a minimisation, and minima are found among the roots of ∇f .

Steps 4–5 are **checks** on the result and on the procedure.

PART A

Solving equations

Root finding and optimisation: what happens inside a fit.

Bisection: slow but certain

If f is continuous and $f(a)f(b) < 0$, a root lies in $[a, b]$.

1. Take the midpoint $m = (a + b)/2$
2. Keep the half where the sign changes
3. Repeat

The bracket halves every step. To reach a tolerance ε :

$$n \approx \log_2 \frac{b - a}{\varepsilon}$$

Trade-off

Guaranteed to converge once you have a bracket.

Slow: one binary digit per step, about 33 steps for 10^{-10} .

Needs only the **sign** of f , not f' .

Newton–Raphson: solving $f(x) = 0$

Start from a guess x_n . Replace f by its tangent line at x_n and find where the tangent crosses zero:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Repeat until

$$|x_{n+1} - x_n| < \varepsilon.$$

Example: Kepler's equation

Where is a planet on its elliptical orbit at time t ?

- $M = 2\pi t/T$: mean anomaly, known from the clock
- e : eccentricity of the orbit
- E : eccentric anomaly, giving the position
 $x = a(\cos E - e)$, $y = b \sin E$

Given M and e , solve for E the equation

$$f(E) = E - e \sin E - M = 0,$$

with $f'(E) = 1 - e \cos E$. No closed form exists, so we iterate.

Newton–Raphson in action

Kepler's equation with $M = 1$, starting from $E_0 = M$. Error $|E_n - E_{\text{exact}}|$:

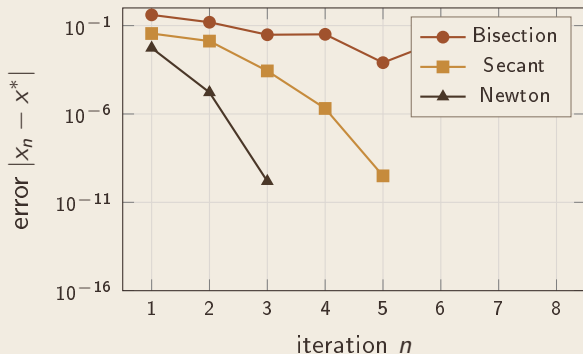
Iteration	$e = 0.5$	$e = 0.9$
1	8×10^{-2}	6×10^{-1}
2	2×10^{-3}	7×10^{-2}
3	6×10^{-7}	2×10^{-3}
4	9×10^{-14}	1×10^{-6}
5	—	4×10^{-13}

Near the root the number of correct digits roughly **doubles** each iteration (quadratic convergence).

It can fail

A poor guess, $f'(x) \approx 0$ or several roots can send it to the wrong root or to infinity. Check that $|f(x)|$ is small at the answer.

Bisection, secant and Newton compared



$f(x) = x^3 - 2x - 5$, root
 $x^* = 2.0945515 \dots$

Bisection	1 digit per step (binary)
Secant	order ≈ 1.6 , no f'
Newton	order 2, needs f'

Speed versus safety

Newton reaches machine precision in 4 steps; bisection is still at 10^{-3} after 8. But only bisection is guaranteed.

Robust root finding in practice

How Newton fails

- $f'(x_n) \approx 0$: a huge jump
- Starting far away: wrong root or divergence
- Cycling between two points
- No bracket: no guarantee a root exists

Brent's method

A hybrid: fast secant / interpolation steps, with a **bisection fallback** whenever a step leaves the bracket.

Guaranteed like bisection, usually as fast as secant.

`scipy.optimize.brentq(f, a, b)`

Bracket first (plot $f!$), then let a fast method finish.

Newton in several dimensions

Solve $F(x) = 0$ for $x \in \mathbb{R}^n$. Replace the derivative by the **Jacobian** $J_{ij} = \partial F_i / \partial x_j$:

$$J(x_n) \Delta x = -F(x_n), \quad x_{n+1} = x_n + \Delta x$$

Example

Intersection of $x^2 + y^2 = 4$ and $y = e^x$:

$$F = \begin{pmatrix} x^2 + y^2 - 4 \\ e^x - y \end{pmatrix}, \quad J = \begin{pmatrix} 2x & 2y \\ e^x & -1 \end{pmatrix}$$

What changes in n dimensions

Each step solves $J\Delta x = -F$; **avoid direct inversion of J** . In 1D a sign change guarantees a root in between; in n dimensions there is no such simple test, so the starting guess decides which root you reach, or whether you reach one.
`scipy.optimize.root`

Optimisation: minima among the roots of the gradient

A minimum of a differentiable f inside its range has $f'(x) = 0$; a root of f' with $f''(x) > 0$ is a local minimum. Newton on f' :

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

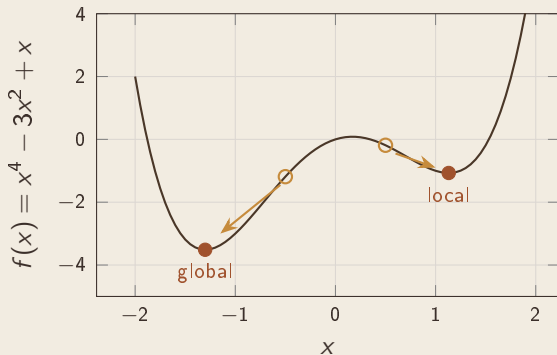
Golden-section search

The bracketing method for minima: keep three points with the lowest in the middle, shrink the interval by 0.618 each step. No derivatives; guaranteed for one minimum in the bracket.

Precision limit

Near a minimum $f(x) \approx f(x^*) + \frac{1}{2}f''(x^*)(x - x^*)^2$. Round-off in f of 10^{-16} limits x to about $\sqrt{10^{-16}} = 10^{-8}$.

Local and global minima



Gradient descent, $x_{n+1} = x_n - \eta f'(x_n)$:

- from $x_0 = -0.5$: $x \rightarrow -1.30$,
 $f = -3.51$
- from $x_0 = +0.5$: $x \rightarrow +1.13$,
 $f = -1.07$

Lesson

Every local method finds a local minimum, not necessarily the global minimum. Nonlinear problems can have several (Lecture 2). Try several starting points.

Minimising in many dimensions

Where $\nabla f = 0$: a local minimum if all eigenvalues of the **Hessian** $H_{ij} = \partial^2 f / \partial x_i \partial x_j$ are > 0 (the many-dimensional $f'' > 0$).

Method	Needs	Character
Nelder–Mead (simplex)	f only	Robust, slow; noisy or non-smooth f
Gradient descent	∇f	Simple; slow in narrow valleys
Quasi-Newton (BFGS)	∇f (builds up H)	The general-purpose default
Newton	∇f , Hessian	Fastest near the minimum; costly

In practice

`scipy.optimize.minimize(f, x0, method=...)`. The choice of x_0 is **yours**, and often matters more than the choice of method.

PART B

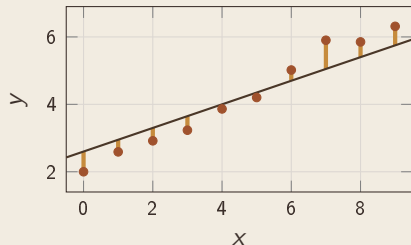
Fitting models

Choose the parameters that make the model closest to the data.

“Closest” must be defined — and it is a choice.

What is a fit?

A model $f(x; \theta)$ with unknown parameters θ , and measurements y_i at x_i , $i = 1, \dots, N$. A **fit** chooses θ so that the model is as close as possible to the data.



A trial line and its **residuals** $r_i = y_i - f(x_i; \theta)$.
“Closest” needs one number. $\sum r_i$ fails: signs cancel (a flat line through the mean gives 0). The sum of squares works:

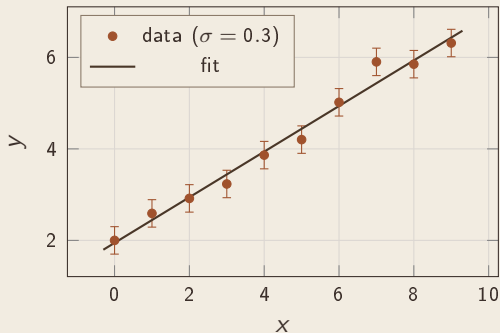
$$S(\theta) = \sum_{i=1}^N r_i^2$$

Least squares: choose $\hat{\theta}$ that minimises S .

When the points have uncertainties σ_i

Measure each residual in units of its standard uncertainty: $\chi^2(\theta) = \sum_i (r_i / \sigma_i)^2$. All σ_i equal: the same $\hat{\theta}$ as S ; unequal: precise points pull harder. For exact x_i and independent Gaussian errors, minimising χ^2 gives the **maximum-likelihood** parameters: those that make the observed data most probable.

The simplest fit: a straight line



Model $y = a + bx$, all $\sigma_i = \sigma$. Setting $\partial\chi^2/\partial a = \partial\chi^2/\partial b = 0$:

$$Na + b \sum x_i = \sum y_i$$

$$a \sum x_i + b \sum x_i^2 = \sum x_i y_i$$

Part A's condition for a minimum; here two **linear** equations: an exact solution, no iteration.

	true	fitted (uncertainty: slide 24)
a	2.0	1.95 ± 0.18
b	0.5	0.499 ± 0.033

$$\chi^2 = 4.2, \nu = N - p = 8$$

Linear least squares in general

“Linear” means linear in the **parameters**, not in x :

$$f(x; \boldsymbol{\theta}) = \sum_{j=1}^p \theta_j \phi_j(x) \quad \text{e.g. } \theta_1 + \theta_2 x + \theta_3 x^2, \quad \theta_1 \sin \omega t + \theta_2 \cos \omega t$$

Design matrix $A_{ij} = \phi_j(x_i)/\sigma_i$, data $b_i = y_i/\sigma_i$:

$$\chi^2 = \|\mathbf{A}\boldsymbol{\theta} - \mathbf{b}\|^2 \quad \Rightarrow \quad \boxed{(\mathbf{A}^\top \mathbf{A}) \hat{\boldsymbol{\theta}} = \mathbf{A}^\top \mathbf{b}}$$

Normal equations

A $p \times p$ linear system: one solve, exact minimum. But solve it with QR/SVD (np.linalg.lstsq): forming $\mathbf{A}^\top \mathbf{A}$ **squares the condition number**, $\kappa(\mathbf{A}^\top \mathbf{A}) = \kappa(\mathbf{A})^2$, where κ is the factor by which relative errors can grow (conditioning, Lecture 1).

Two meanings of “linear”

	Linear dynamics (Lecture 2)	Linear in parameters (today)
Property of	the system	the fitting function
Meaning	superposition; frequency independent of amplitude	$f = \sum_j \theta_j \phi_j(x)$
How to tell	test it: superposition, amplitude dependence, harmonics, surrogate data	read it off the formula
Consequence	what behaviour to expect	exact solve, or iterate from starting values

They are independent

Harmonic oscillator, $x = A \sin(\omega t + \varphi)$: **linear dynamics**, but fitting an unknown ω is a **nonlinear fit**.
Van der Pol limit cycle: **nonlinear dynamics**, but its Fourier series with known period is a **linear fit**.

Nonlinear models

Example: the stirred-reactor relaxation of Lecture 2, $C(t) = C_{ss} + (C_0 - C_{ss})e^{-t/\tau}$: initial concentration C_0 , steady state C_{ss} , relaxation time τ . With τ inside the exponential, $\chi^2(\theta)$ is not a quadratic bowl: **iterate**.

Gauss–Newton: Newton for χ^2

1. Guess θ
2. Tangent:
$$f(x_i; \theta + \delta) \approx f(x_i; \theta) + \sum_j J_{ij} \delta_j,$$
$$J_{ij} = \partial f(x_i; \theta) / \partial \theta_j$$
3. Solve this *linear* least-squares problem for δ (slides 16–17)
4. $\theta \rightarrow \theta + \delta$; repeat until χ^2 stops decreasing

Levenberg–Marquardt: shorter, more down-hill steps far from the minimum.

In Python

```
def model(t, Css, C0, tau):  
    return Css + (C0-Css)*np.exp(-t/tau)  
  
theta, cov = curve_fit(model, t, y,  
    p0=(1, 0, 3),          # Css, C0, tau  
    sigma=sig,             # errors of y  
    absolute_sigma=True)   # keep sig  
err = np.sqrt(np.diag(cov))
```

Needs a **starting guess** p_0 : local minima again (Part A). Without `absolute_sigma=True`, σ is rescaled so that $\chi^2/\nu = 1$, hiding a bad fit.

PART C

Is the fit any good?

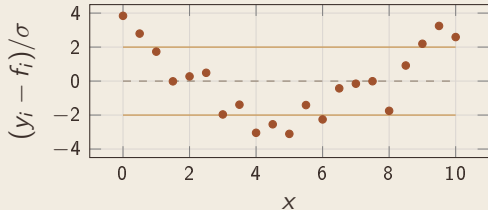
A fitting routine always returns numbers.
Whether they mean anything is your job.

Look at the residuals first

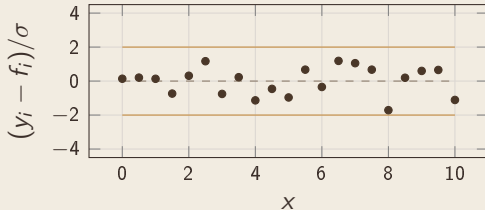
Problem: a fit always returns “best” parameters, even when the model has the wrong form. Is the shape of the model right? χ^2 is one number; the residuals show *where* and *how* the model fails.

Data: slightly curved, $y = 1 + 0.2x + 0.03x^2$, 21 points, all with $\sigma = 0.15$; fitted with a line (left) and a quadratic (right). Residuals in units of σ :

Straight line: $\chi^2/\nu = 4.75$



Quadratic: $\chi^2/\nu = 0.76$



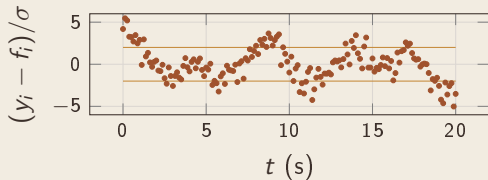
The line leaves a **systematic arch**: the model has the wrong shape. The quadratic leaves structureless scatter, mostly within ± 2 .

Residuals: time variability before noise

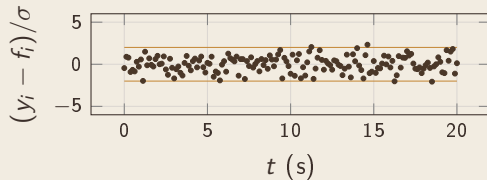
Data: $y_i = \sin(\omega_0 t_i + \frac{1}{2}\beta t_i^2) + \varepsilon_i$, $\omega_0 = 1$ rad/s, $\beta = 0.015$ rad/s², 161 points for $0 \leq t \leq 20$ s, all with $\sigma = 0.1$. The frequency $\omega(t) = \omega_0 + \beta t$ drifts from 1.0 to 1.3 rad/s.

Left: $f(t) = A \sin(\omega t + \varphi)$, constant ω (3 parameters). Right: $f(t) = A \sin(\omega_0 t + \frac{1}{2}\beta t^2 + \varphi)$ (4 parameters).

Constant ω : $\chi^2/\nu = 3.97$



Drifting $\omega(t) = \omega_0 + \beta t$: $\chi^2/\nu = 0.91$



Right fit: $\beta = 0.0153 \pm 0.0007$ rad/s² (true 0.015), $\omega_0 = 0.994 \pm 0.007$ rad/s (true 1).

The tempting fix is wrong

Inflating σ to 0.20 makes the constant- ω fit “acceptable” — and turns deterministic **time variability** into “random error”, doubling every error bar.

The reduced chi-square χ^2/ν

If the model is right and the σ_i are right, each term of χ^2 is about 1, so

$$\boxed{\frac{\chi^2}{\nu} \approx 1 \pm \sqrt{2/\nu}}, \quad \nu = N - p.$$

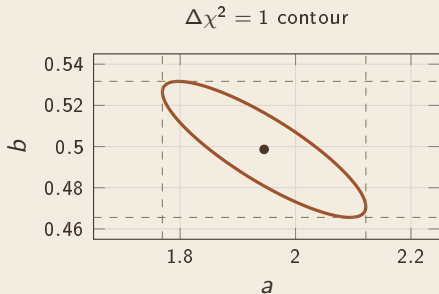
-
- | | |
|------------------------|--|
| $\chi^2/\nu \gg 1$ | Wrong model (often unmodelled time variability) or σ_i underestimated. Look at the residuals first; only if they show no structure may you conclude the σ_i are too small. |
| $\chi^2/\nu \approx 1$ | Consistent: model and uncertainties agree with the data. |
| $\chi^2/\nu \ll 1$ | σ_i overestimated, or overfitting . Suspicious, not “excellent”. |
-

Examples so far: drifting ω fitted as constant 3.97; line on curved data 4.75; quadratic on curved data 0.76; line on straight data (slide 16) 0.52, with $\nu = 8$ (range 1 ± 0.5).

Parameter uncertainties and correlations

Linear model: covariance matrix $C = (A^T A)^{-1}$, the inverse of the normal-equation matrix (curve_fit returns C for nonlinear fits):

$$\sigma_{\theta_j} = \sqrt{C_{jj}}, \quad \rho_{jk} = \frac{C_{jk}}{\sqrt{C_{jj} C_{kk}}}.$$

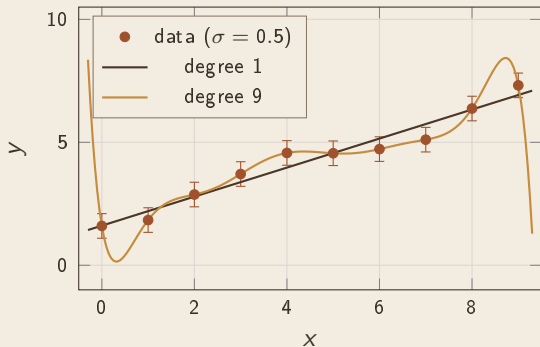


Straight-line example

$a = 1.95 \pm 0.18$, $b = 0.499 \pm 0.033$, $\rho_{ab} = -0.84$: the tilted ellipse. Its shadows on the axes (dashed) are the $\pm 1\sigma$ ranges.

Consequence: $a \pm \sigma_a$ and $b \pm \sigma_b$ alone misstate what you know about combinations. Centring x ($x - \bar{x}$) removes the correlation.

Overfitting



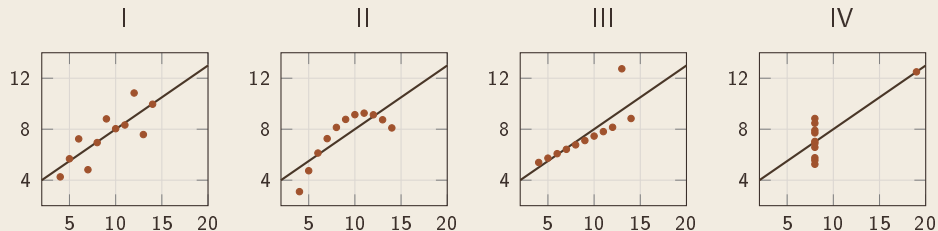
10 points from a straight line plus noise.

degree	ν	χ^2/ν
1	8	0.67
3	6	0.55
6	3	0.06
9	0	— ($\chi^2 = 0$)

Prediction at $x = 10$

Line: 7.5. Degree 9: -72.

Anscombe's quartet: numbers are not enough



All four: mean $\bar{x} = 9$, $\bar{y} = 7.50$, fitted line $y = 3.00 + 0.500x$, $R^2 = 0.67$. Only one is a sensible straight-line fit.

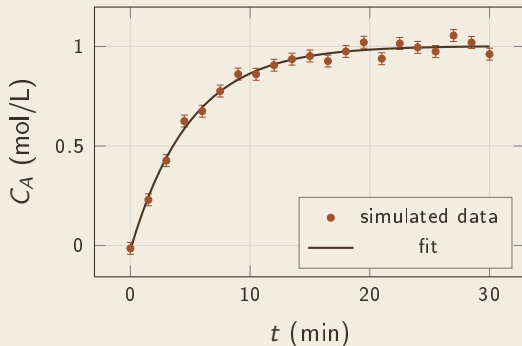
Plot the data and the residuals before you trust a single summary number.

PART D

Generated data and measured data

First test the fitting procedure where you know the answer.
Then use it where you do not.

Generated data: can we recover what we put in?



CSTR relaxation from Lecture 2, plus noise $\sigma = 0.03$, $N = 21$.

	true	fitted
C_{ss}	1.0	1.003 ± 0.011
C_0	0.0	-0.021 ± 0.026
τ	5.0	5.00 ± 0.27

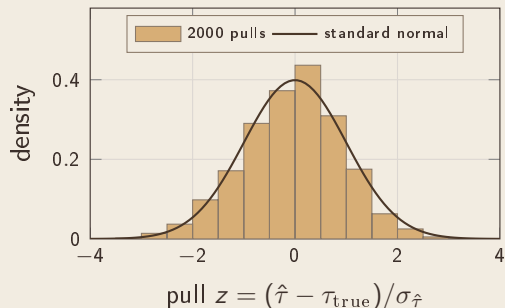
$$\chi^2/\nu = 0.93$$

Verification of the fit

All three true values lie within 1σ . The procedure works — for *this* realisation.

Are the error bars right?

Problem: the previous fit quotes $\tau = 5.00 \pm 0.27$ min. Would repeated experiments really scatter by ± 0.27 ?



Test by Monte Carlo (Lecture 2):

1. New data set: same reactor ($\tau_{\text{true}} = 5$ min), new noise
2. Fit: $\hat{\tau}$ and its quoted uncertainty $\sigma_{\hat{\tau}}$
3. **Pull** $z = (\hat{\tau} - \tau_{\text{true}})/\sigma_{\hat{\tau}}$: the error in units of its own quoted uncertainty
4. Repeat 2000 times

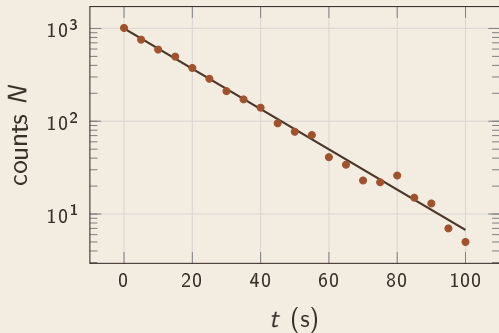
Right error bars and no bias $\Rightarrow z$ is standard normal:

	expected	found
mean	0	-0.04
std	1	0.97
$ z < 1$	68%	70%

Here: no bias; the quoted $\sigma_{\hat{\tau}}$ are honest.

Warning signs: spread of z above 1 – error bars too small; mean away from 0 – a bias.

Counting data: the noise model matters



Radioactive decay, $N = N_0 e^{-t/\tau}$ (simulated here, so that τ is known). Counts are

Poisson: $\sigma_i = \sqrt{N_i}$, not constant. Late points: 15, 13, 7, 5 counts.

Method (2000 data sets, $\tau = 20$ s)	spread of $\hat{\tau}$
Nonlinear fit, weights $1/\sqrt{N_i}$	± 0.34 s
Straight line through $\ln N$, unweighted	± 0.68 s

Lesson

“Linearising” with logs and ignoring weights lets the noisiest points count as much as the best ones: same data, **twice** the uncertainty.

What statistics cannot see

Statistical uncertainty

Random scatter. Shrinks as $1/\sqrt{N}$. Detected by residuals, χ^2 , pulls.

Systematic error

A wrong calibration or assumption **common to all points**. Does **not** shrink with more data. Invisible to χ^2 .

Hubble, 1929

A clean linear fit gave $H_0 \approx 500 \text{ km s}^{-1} \text{ Mpc}^{-1}$; today about 70. The distances were systematically wrong — more data would only have made the wrong answer more precise.

A fitting workflow

	Step	Tool / check
1	Plot the data; classify the system (Lecture 2)	Expected shape, noise type
2	Choose the simplest physical model	Linear in θ ? (not the same as linear dynamics)
3	Test on generated data with known θ	Recover θ ; pulls $\sim \mathcal{N}(0, 1)$
4	Fit the measured data	lstsq / curve_fit, honest σ_i , good p0
5	Judge the fit	Residuals: time variability before noise; χ^2/ν
6	Report	$\hat{\theta} \pm \sigma$, correlations, χ^2/ν , systematics

Before you fit: five questions

1. What kind of system produced the data? (*linear or nonlinear dynamics; time-varying?*)
2. Is my fitting function linear in its parameters? (*exact solve, or iterate from starting values*)
3. What is the noise, and do I know σ_i ? (*Gaussian? Poisson? constant?*)
4. Can I recover known parameters from simulated data? (*verification*)
5. Is there time variability left in the residuals, before I call it noise? (*goodness of fit*)

Then report the parameters, their uncertainties and their correlations.

TO TAKE AWAY

A fit always returns numbers.

Know the system, choose the model, test on known data, look for time variability in the residuals — then believe the numbers.